

Capitolul 2

Rezolvarea ecuațiilor

2.1 Metoda biseției

Codul MATLAB pentru metoda biseției este următorul:

Algoritm (Metoda biseției).

Dându-se un interval inițial $[a, b]$ astfel încât $f(a)f(b) < 0$

***while** $(b - a)/2 > TOL$*

$c = (a + b)/2$

if** $f(c) = 0$, **stop**, **end

***if** $f(a)f(c) < 0$*

$b = c$

else

$a = c$

end

end

Intervalul final $[a, b]$ conține o rădăcină. Aproximarea rădăcinii este $(a + b)/2$.

```
function xc=bisect(f,a,b,tol)
fa=f(a);
fb=f(b);
if sign(fa)*sign(fb) >= 0
    error('f(a)f(b)<0 not satisfied!')
end
while (b-a)/2>tol
    c=(a+b)/2;
    fc=f(c);
    if fc == 0
        break;
    end
    if sign(fc)*sign(fa)<0
        b=c;
        fb=fc;
    else
        a=c;
        fa=fc;
    end
end
xc=(a+b)/2;
```

Pentru a folosi funcția `bisect.m`, definim o funcție MATLAB astfel:

```
>> f=@(x) x^3+x-1;
```

Această comandă definește un function handle `f`, care poate fi folosit ca intrare pentru alte funcții MATLAB. Atunci comanda

```
>> xc=bisect(f,0,1,0.00005)
```

returnează o soluție care este corectă cu o toleranță de 0.00005.

2.2 Iterația de punct fix

Algoritmul pentru iterația de punct fix aplicată unei funcții `g` poate fi scris în MATLAB astfel:

Algorithm (Iterația de punct fix).

$x_0 = \text{valoarea inițială}$

$x_{i+1} = g(x_i)$ **for** $i = 0, 1, 2, \dots$

```
function xc=fpi(g,x0,k)
x(1)=x0;
for i=1:k
    x(i+1)=g(x(i));
end
xc=x(k+1);
```

După definirea unei funcții MATLAB prin

```
>> g=@(x) (1+2*x^3)/(1+3*x^2);
```

funcția de mai sus poate fi apelată astfel:

```
>> xc=fpi(g,0,10)
```

pentru a rula 10 pași din iterația de punct fix cu valoarea inițială 0.

2.3 Metoda lui Newton

Codul metodei lui Newton în MATLAB este:

Algorithm (Metoda lui Newton).

$x_0 = \text{valoarea inițială}$

$x_{i+1} = x_i - \frac{f(x_i)}{f'(x_i)}$ **for** $i = 0, 1, 2, \dots$

```
function xc=newton(f,df,x0,k)
x(1)=x0;
for i=1:k
    x(i+1)=x(i)-f(x(i))/df(x(i));
end
xc=x(k+1);
```

Pentru a folosi funcția `newton.m`, definim un function handle pentru `f`

```
>> f=@(x) x^3+x-1;
```

și unul pentru derivata lui f :

```
>> df=@(x) 3*x^2+1;
```

Atunci comanda

```
>> xc=newton(f,df,0,10)
```

rulează 10 pași din metoda lui Newton cu valoarea inițială 0.

2.4 Găsirea rădăcinilor fără derivate

Comanda MATLAB `fzero` implementează o versiune a metodei lui Brent, împreună cu pas de preprocesare care caută să găsească un interval inițial în care să caute soluția, în cazul în care un astfel de interval nu este dat de către utilizator.

Următorul exemplu de folosire a funcției `fzero` definește funcția $f(x) = x^3 + x - 1$ și intervalul $[0, 1]$ în care va fi căutată rădăcina lui f :

```
>> f=@(x) x^3+x-1;
>> fzero(f,[0 1])
```

Ca o alternativă, comanda

```
>> fzero(f,1)
```

caută o rădăcină a lui f într-o vecinătate a lui $x = 1$, găsind mai întâi un interval de căutare și aplicând apoi metoda lui Brent.

Probleme

1. Folosiți metoda biseției pentru a localiza toate rădăcinile ecuației $2x^3 - 6x - 1 = 0$. Reprezentați grafic funcția utilizând `ezplot` și identificați trei intervale de lungime 1 care conțin o rădăcină. Găsiți apoi fiecare dintre cele trei rădăcini cu șase zecimale exacte.

```
-1.641783237457275
-0.168254375457764
1.810038089752197
```

2. Implementați în MATLAB metoda secantei într-o funcție care să aibă următoarea semnătură:

```
function xc=secant(f,x0,x1,k)
```

unde x_0, x_1 sunt valorile inițiale, f este function handle-ul și k numărul de pași.

Algorithm (Metoda secantei).

$x_0, x_1 = \text{valorile inițiale}$

$x_{i+1} = x_i - \frac{f(x_i)(x_i - x_{i-1})}{f(x_i) - f(x_{i-1})}$ **for** $i = 1, 2, 3, \dots$

3. Aplicați 10 pași ai iterației de punct fix pentru a determina soluția ecuației $x^5 + x - 1 = 0$, cu valoarea inițială 0.8.

0.755250803461252

4. Găsiți soluția ecuației $x^3 - 2x - 2 = 0$ folosind 10 iterații ale metodei lui Newton.

1.769292354238631

5. Implementați într-o funcție MATLAB metoda falsei poziții, care să aibă semnătura:

```
function xc=mfp(f,a,b,k)
```

unde a și b sunt capetele intervalului de căutare, f este function handle-ul și k numărul de pași.

Algoritm (Metoda falsei poziții).

Dându-se intervalul $[a, b]$ astfel încât $f(a)f(b) < 0$

for $i = 1, 2, 3, \dots$

$$c = \frac{bf(a) - af(b)}{f(a) - f(b)}$$

if $f(c) = 0$, stop, end

if $f(a)f(c) < 0$

$$b = c$$

else

$$a = c$$

end

end

6. Aplicați metoda secantei și metoda falsei poziții pentru cele 3 ecuații de la Problemele 1, 3 și 4, și comparați rezultatele cu cele obținute folosind funcția **fzero** din MATLAB.

-1.641782244880795

-1.641782805464868

-1.641783527452926

-0.168254401781027

-0.168254401781027

-0.168254401781027

1.810037929233953

1.810037928808139

1.810037929233953

0.754877663184338

0.754868311712477

0.754877666246693

1.769292354238631

1.769292353338026

1.769292354238632